

Binary Search Tree Implementation

Subject: Data Structures Topic: Trees

Overview

BST properties, insertion, deletion, and balancing techniques

BST invariant

- For each node, values in its left subtree are smaller and values in its right subtree are larger, subject to the chosen duplicate policy.
- Search, insert, and delete take $O(h)$, where h is tree height; a skewed tree can have $h = n$.
- An in-order traversal returns keys in sorted order and runs in $O(n)$.

Operations

- Search follows one branch at each comparison until a matching key or null is reached.
- Deleting a leaf removes it directly; deleting a node with one child links its parent to that child.
- For two children, replace with the in-order successor (minimum of the right subtree) or predecessor, then remove that replacement node.

Balancing

- A balanced tree keeps height $O(\log n)$, giving logarithmic search and update operations.
- AVL trees maintain a strict height-balance condition and use rotations after updates.
- Red-black trees enforce color invariants that bound height while requiring fewer rotations in many workloads.

Testing

- Test empty trees, root deletion, missing keys, duplicate keys, and ascending insertion order.
- After every mutation, verify the ordering invariant and the node count.
- Use iterative traversal when recursion depth could exceed the runtime stack.